Artificial Intelligence: Tutorial-9

Name: Gaurav Damor

Admission Number: U24AI026

1. Implement the Min-Max algorithm to solve a simple two-player Tic-Tac-Toe evaluate its performance, and visualize the search tree.

CODE:

```
board = [
    ['-', '-', '-'],
    ['-', '-', '-'],
    ['-', '-', '-']
]

states = 0

def print_b(b):
    for r in b:
        print(r[0] + " " + r[1] + " " + r[2])
    print("")

def check_win(b):
    if b[0][0] == b[0][1] and b[0][1] == b[0][2] and b[0][0] != '-': return b[0][0]
    if b[1][0] == b[1][1] and b[1][1] == b[1][2] and b[1][0] != '-': return b[1][0]
    if b[2][0] == b[2][1] and b[2][1] == b[2][2] and b[2][0] != '-': return b[2][0]

    if b[0][0] == b[1][0] and b[1][0] == b[2][0] and b[0][0] != '-': return b[0][0]
    if b[0][1] == b[1][1] and b[1][1] == b[2][1] and b[0][1] != '-': return b[0][1]
    if b[0][2] == b[1][2] and b[1][2] == b[2][2] and b[0][2] != '-': return b[0][2]

    if b[0][0] == b[1][1] and b[1][1] == b[2][2] and b[0][0] != '-': return b[0][0]
    if b[0][2] == b[1][1] and b[1][1] == b[2][0] and b[0][2] != '-': return b[0][2]

    empty = 0
    for i in [0, 1, 2]:
        for j in [0, 1, 2]:
            if b[i][j] == '-':
                empty = empty + 1
    if empty == 0:
        return 'Tie'

    return 'None'

def minimax(b, depth, is_max):
    global states
    states = states + 1
```

```
    ind = ""
    for x in range(depth):
        ind = ind + "    "
    ind = ind + "|-- "

    res = check_win(b)
    if res == 'X':
        return 10 - depth
    elif res == 'O':
        return -10 + depth
    elif res == 'Tie':
        return 0

    if is_max == True:
        best = -1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'X'
                    print(ind + "X -> " + str(i) + "," + str(j))
                    val = minimax(b, depth + 1, False)
                    b[i][j] = '-'
                    if val > best:
                        best = val
        return best
    else:
        best = 1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'O'
                    print(ind + "O -> " + str(i) + "," + str(j))
                    val = minimax(b, depth + 1, True)
                    b[i][j] = '-'
                    if val < best:
                        best = val
        return best

def find_best(b):
    global states
    states = 0
    best_val = -1000
    best_r = -1
    best_c = -1

    print("Tree:")

    for i in [0, 1, 2]:
        for j in [0, 1, 2]:
            if b[i][j] == '-':
                b[i][j] = 'X'
                print("|-- X -> " + str(i) + "," + str(j))
                move_val = minimax(b, 1, False)
                b[i][j] = '-'
```

```
                if move_val > best_val:
                    best_r = i
                    best_c = j
                    best_val = move_val

    print("\nStates: " + str(states))
    return [best_r, best_c]

# Check on board
board[0][0] = 'O'
board[0][1] = 'X'
board[0][2] = 'O'
board[1][0] = '-'
board[1][1] = '-'
board[1][2] = '-'
board[2][0] = '-'
board[2][1] = '-'
board[2][2] = '-'

print("Board:")
print_b(board)

ans = find_best(board)
print("Best Move: " + str(ans[0]) + "," + str(ans[1]))
```

OUTPUT:

Initial Board:

```
 |  |

 |  |

 |  |

---------
```

Node 1 (Depth 0):

```
X |  |

 |  |

 |  |

---------
```

Node 2 (Depth 1):

```
X | O |
  |   |
  |   |
---------
```

Node 3 (Depth 2):

```
X | O | X
  |   |
  |   |
---------
```

Node 4 (Depth 3):

```
X | O | X
O |   |
  |   |
---------
```

Node 5 (Depth 4):

```
X | O | X
O | X |
  |   |
---------
```

Node 6 (Depth 5):

```
X | O | X
O | X | O
  |   |
---------
```

Node 7 (Depth 6):

X | O | X

O | X | O

X | |

Node 49 (Depth 7):

X | O | X

O | O | X

X | O |

Node 50 (Depth 7):

X | O | X

O | O | X

X | | O

---

2. Extend Lab 1 with Alpha-Beta pruning, compare efficiency gains, and analyze pruning patterns on varied game trees.

CODE:

```
states_mm = 0
states_ab = 0

def print_b(b):
    for r in b:
        print(r[0] + " " + r[1] + " " + r[2])
    print("")

def check_win(b):
    if b[0][0] == b[0][1] and b[0][1] == b[0][2] and b[0][0] != '-': return b[0][0]
    if b[1][0] == b[1][1] and b[1][1] == b[1][2] and b[1][0] != '-': return b[1][0]
    if b[2][0] == b[2][1] and b[2][1] == b[2][2] and b[2][0] != '-': return b[2][0]

    if b[0][0] == b[1][0] and b[1][0] == b[2][0] and b[0][0] != '-': return b[0][0]
    if b[0][1] == b[1][1] and b[1][1] == b[2][1] and b[0][1] != '-': return b[0][1]
    if b[0][2] == b[1][2] and b[1][2] == b[2][2] and b[0][2] != '-': return b[0][2]

    if b[0][0] == b[1][1] and b[1][1] == b[2][2] and b[0][0] != '-': return b[0][0]
```

```
    if b[0][2] == b[1][1] and b[1][1] == b[2][0] and b[0][2] != '-': return b[0][2]

    empty = 0
    for i in [0, 1, 2]:
        for j in [0, 1, 2]:
            if b[i][j] == '-':
                empty = empty + 1
    if empty == 0:
        return 'Tie'

    return 'None'

def minimax(b, depth, is_max):
    global states_mm
    states_mm = states_mm + 1

    res = check_win(b)
    if res == 'X': return 10 - depth
    elif res == 'O': return -10 + depth
    elif res == 'Tie': return 0

    if is_max == True:
        best = -1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'X'
                    val = minimax(b, depth + 1, False)
                    b[i][j] = '-'
                    if val > best: best = val
        return best
    else:
        best = 1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'O'
                    val = minimax(b, depth + 1, True)
                    b[i][j] = '-'
                    if val < best: best = val
        return best

def alphabeta(b, depth, is_max, a, be):
    global states_ab
    states_ab = states_ab + 1

    ind = ""
    for x in range(depth):
        ind = ind + "    "
    ind = ind + "|-- "

    res = check_win(b)
    if res == 'X': return 10 - depth
    elif res == 'O': return -10 + depth
```

```
    elif res == 'Tie': return 0

    if is_max == True:
        best = -1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'X'
                    print(ind + "X -> " + str(i) + "," + str(j))
                    val = alphabeta(b, depth + 1, False, a, be)
                    b[i][j] = '-'
                    if val > best: best = val
                    if best > a: a = best
                    if be <= a:
                        print(ind + "Pruned!")
                        return best
        return best
    else:
        best = 1000
        for i in [0, 1, 2]:
            for j in [0, 1, 2]:
                if b[i][j] == '-':
                    b[i][j] = 'O'
                    print(ind + "O -> " + str(i) + "," + str(j))
                    val = alphabeta(b, depth + 1, True, a, be)
                    b[i][j] = '-'
                    if val < best: best = val
                    if best < be: be = best
                    if be <= a:
                        print(ind + "Pruned!")
                        return best
        return best

def test_board(b, name):
    global states_mm
    global states_ab
    states_mm = 0
    states_ab = 0

    print("------- " + name + " -------")
    print_b(b)

    best_val_mm = -1000
    for i in [0, 1, 2]:
        for j in [0, 1, 2]:
            if b[i][j] == '-':
                b[i][j] = 'X'
                move_val = minimax(b, 0, False)
                b[i][j] = '-'
                if move_val > best_val_mm:
                    best_val_mm = move_val

    print("Tree built by Alpha-Beta:")
    best_val_ab = -1000
```

```
    a = -1000
    be = 1000
    for i in [0, 1, 2]:
        for j in [0, 1, 2]:
            if b[i][j] == '-':
                b[i][j] = 'X'
                print("|-- X -> " + str(i) + "," + str(j))
                move_val = alphabeta(b, 1, False, a, be)
                b[i][j] = '-'
                if move_val > best_val_ab:
                    best_val_ab = move_val
                if best_val_ab > a:
                    a = best_val_ab

    print("")
    print("Minimax States   : " + str(states_mm))
    print("AlphaBeta States : " + str(states_ab))
    print("")

board1 = [
    ['X', 'O', 'X'],
    ['O', '-', '-'],
    ['-', '-', '-']
]

board2 = [
    ['-', 'O', '-'],
    ['X', '-', 'X'],
    ['-', '-', 'O']
]

test_board(board1, "Board 1")
test_board(board2, "Board 2")
```

OUTPUT:

------- Board 1 -------

X O X

O - -

- - -

Tree built by Alpha-Beta:

|-- X -> 1,1

  |-- O -> 1,2

```
    |-- X -> 2,0
    |-- X -> 2,1
        |-- O -> 2,0
            |-- X -> 2,2
        |-- Pruned!
    |-- X -> 2,2
|-- O -> 2,0
    |-- X -> 1,2
        |-- O -> 2,1
            |-- X -> 2,2
        |-- O -> 2,2
            |-- X -> 2,1
    |-- X -> 2,1
        |-- O -> 1,2
            |-- X -> 2,2
        |-- O -> 2,2
            |-- X -> 1,2
        |-- Pruned!
    |-- X -> 2,2
    |-- Pruned!
|-- O -> 2,1
    |-- X -> 1,2
        |-- O -> 2,0
            |-- X -> 2,2
        |-- O -> 2,2
            |-- X -> 2,0
            |-- Pruned!
    |-- X -> 2,0
    |-- Pruned!
```

```
    |-- O -> 2,2
        |-- X -> 1,2
            |-- O -> 2,0
                |-- X -> 2,1
            |-- O -> 2,1
                |-- X -> 2,0
                |-- Pruned!
        |-- X -> 2,0
        |-- Pruned!
|-- X -> 1,2
    |-- O -> 1,1
        |-- X -> 2,0
            |-- O -> 2,1
            |-- Pruned!
        |-- X -> 2,1
            |-- O -> 2,0
                |-- X -> 2,2
            |-- Pruned!
        |-- X -> 2,2
    |-- Pruned!
|-- X -> 2,0
    |-- O -> 1,1
        |-- X -> 1,2
            |-- O -> 2,1
            |-- Pruned!
        |-- X -> 2,1
            |-- O -> 1,2
            |-- Pruned!
        |-- X -> 2,2
```

```
        |-- O -> 1,2
        |-- Pruned!
    |-- Pruned!
|-- X -> 2,1
    |-- O -> 1,1
        |-- X -> 1,2
            |-- O -> 2,0
                |-- X -> 2,2
            |-- Pruned!
        |-- X -> 2,0
            |-- O -> 1,2
            |-- Pruned!
        |-- X -> 2,2
            |-- O -> 1,2
            |-- Pruned!
    |-- Pruned!
|-- X -> 2,2
    |-- O -> 1,1
        |-- X -> 1,2
        |-- X -> 2,0
            |-- O -> 1,2
            |-- Pruned!
        |-- X -> 2,1
            |-- O -> 1,2
            |-- Pruned!
    |-- Pruned!
```

Minimax States : 233

AlphaBeta States : 65

------- Board 2 -------

- O -

X - X

- - O

Tree built by Alpha-Beta:

```
|-- X -> 0,0
    |-- O -> 0,2
        |-- X -> 1,1
        |-- X -> 2,0
        |-- X -> 2,1
            |-- O -> 1,1
                |-- X -> 2,0
            |-- Pruned!
    |-- O -> 1,1
        |-- X -> 0,2
            |-- O -> 2,0
                |-- X -> 2,1
            |-- O -> 2,1
        |-- X -> 2,0
        |-- Pruned!
    |-- O -> 2,0
        |-- X -> 0,2
            |-- O -> 1,1
                |-- X -> 2,1
            |-- O -> 2,1
        |-- X -> 1,1
        |-- Pruned!
```

```
    |-- O -> 2,1
        |-- X -> 0,2
            |-- O -> 1,1
            |-- O -> 2,0
        |-- X -> 1,1
        |-- Pruned!
|-- X -> 0,2
    |-- O -> 0,0
        |-- X -> 1,1
        |-- X -> 2,0
            |-- O -> 1,1
            |-- Pruned!
        |-- X -> 2,1
            |-- O -> 1,1
            |-- Pruned!
    |-- Pruned!
|-- X -> 1,1
|-- X -> 2,0
    |-- O -> 0,0
        |-- X -> 0,2
            |-- O -> 1,1
            |-- Pruned!
        |-- X -> 1,1
        |-- X -> 2,1
            |-- O -> 0,2
            |-- Pruned!
    |-- Pruned!
|-- X -> 2,1
    |-- O -> 0,0
```

```
        |-- X -> 0,2
            |-- O -> 1,1
            |-- Pruned!
        |-- X -> 1,1
        |-- X -> 2,0
            |-- O -> 0,2
            |-- Pruned!
    |-- Pruned!
```

Minimax States : 161

AlphaBeta States : 46